



UNIVERSIDADE FEDERAL DE MINAS GERAIS

INSTITUTO DE CIÊNCIAS EXATAS

DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

Trabalho Prático I: Análise Dinâmica de Carteiras de Ações

Thais Sá

Matrícula: 2025049441

E-mail: thais-sa@ufmg.br

Disciplina: Estruturas de Dados

Professor: Marcio Costa Santos
DCC/UFMG

Belo Horizonte
2026

Sumário

1	Introdução	2
2	Método	2
2.1	Estruturas de Dados	2
2.2	Algoritmo	3
2.3	Classes	3
3	Análise de Complexidade	4
3.1	Complexidade de Tempo	4
3.2	Complexidade de Espaço	5
4	Estratégias de Robustez	5
5	Análise Experimental	6
5.1	Ambiente e Metodologia de Testes	6
5.2	Resultados e Discussão	6
6	Conclusão	8
7	Bibliografia	9

1 Introdução

O problema proposto foi o desenvolvimento de um sistema de recomendações financeiras para a empresa fictícia Jolambs, cujo objetivo era aprimorar a análise dinâmica de carteiras de ações. O sistema deve processar um fluxo de eventos em tempo real, mantendo o histórico de cotações de um conjunto de ações, gerenciando as carteiras de investimentos dos clientes por meio de compras e vendas, e respondendo a consultas de desempenho que combinam múltiplas métricas financeiras, gerando rankings de forma contínua.

O desafio central consiste em responder a consultas que retornam as n melhores e n piores ações da carteira de um cliente. Esse ranqueamento é baseado em uma combinação ponderada de métricas financeiras predefinidas — Retorno Acumulado (RET), Média de Retornos (AVGRET), Estabilidade (STAB) e Consistência de Retornos (CONS) — cujos pesos são definidos pelo usuário em cada consulta. Por exemplo, um cliente com perfil conservador pode atribuir maior peso à métrica STAB, priorizando ações com menor volatilidade.

O grande desafio algorítmico reside no custo computacional de manter uma ordenação global de todas as ações cadastradas para cada métrica, visto que novas cotações alteram dinamicamente essas posições. Este trabalho investiga duas estratégias distintas para lidar com esse compromisso: a **Estratégia Sob Demanda**, que adia o cálculo do ranking até o momento da consulta, e a **Estratégia Imediata**, que mantém as ordenações sempre atualizadas após cada nova cotação.

2 Método

O programa foi desenvolvido na linguagem C++11 e compilado via GNU Compiler Collection (g++). Para a análise experimental, utilizou-se a linguagem Python em um *framework* no Jupyter Notebook para geração das entradas e plotagem dos gráficos de complexidade temporal.

2.1 Estruturas de Dados

A base da gestão de memória do sistema é a implementação de **vetores dinâmicos** construídos manualmente, sem uso das estruturas da biblioteca padrão do C++. Como o número de ações, clientes e cotações não é conhecido antes da execução, adotou-se a estratégia de alocação dinâmica com redimensionamento dobrado: sempre que a capacidade máxima de um vetor é atingida, um novo vetor com o dobro da capacidade é alocado, os dados são copiados e a memória antiga é liberada com `delete[]`. Esta abordagem garante que a inserção de novos elementos tenha um custo amortizado de $O(1)$ — embora operações individuais de redimensionamento custem $O(n)$, esse custo se dilui sobre a sequência de inserções.

A Figura 1 ilustra esquematicamente a estrutura interna de um vetor dinâmico, tomando como exemplo o vetor de cotações da classe `Acao`.

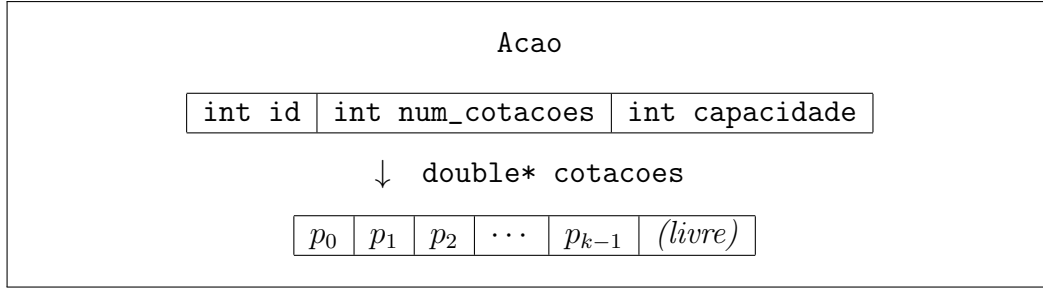


Figura 1: Estrutura interna do vetor dinâmico de cotações da classe **Acao**. O ponteiro **cotacoes** aponta para um bloco contínuo de memória. Quando **num_cotacoes** atinge **capacidade**, o vetor é redimensionado, e passa a ter o dobro da capacidade antiga.

2.2 Algoritmo

Para o ranqueamento das métricas, implementou-se o algoritmo de ordenação **QuickSort**. A escolha do QuickSort justifica-se pela sua eficiência de tempo no caso médio de $O(n \log n)$ e por ser um algoritmo do tipo *in-place*, minimizando alocações adicionais de memória durante as consultas. O algoritmo foi implementado de forma iterativa — com uma pilha manual em vez de recursão — para evitar estouro de pilha em entradas com muitas ações. O pivô é escolhido como o **elemento central** do subarray (**arr[meio]**), o que melhora o desempenho em arrays parcialmente ordenados e reduz as chances de ocorrer o pior caso do QuickSort (o caso quadrático).

O critério de comparação do QuickSort foi adaptado para ordenar um array de structs auxiliares (**ParOrdenacao**), priorizando pontuação em ordem decrescente e, em caso de empate, o identificador da ação em ordem crescente.

2.3 Classes

Para modularizar a implementação, foram desenvolvidas três classes principais e uma *struct* auxiliar:

- **Acao:** Classe responsável por encapsular o histórico de preços da ação. Ela armazena as cotações em um vetor dinâmico interno e possui métodos específicos para calcular as métricas (**RET**, **AVGRET**, **STAB** e **CONS**) sobre as últimas w cotações disponíveis.
- **Cliente:** Classe que gerencia a carteira individual de investimentos. Ela mantém um vetor dinâmico de inteiros com os IDs das ações possuídas. A classe tem os métodos para adicionar (compra) e remover uma ação (venda, com complexidade $O(c)$ para busca e $O(1)$ para remoção, onde c é o tamanho da carteira). A remoção é otimizada: o elemento a ser removido é substituído pelo último do vetor, evitando deslocamentos.
- **ParOrdenacao:** Uma *struct* leve contendo apenas um **double** **pontuacao** e um **int** **id**. O uso desta estrutura evita a cópia de objetos pesados durante as trocas do QuickSort, otimizando o tempo de resposta nas ordenações.
- **Ranking:** Implementa a função **QuickSort** e utiliza a struct **ParOrdenacao**.

3 Análise de Complexidade

3.1 Complexidade de Tempo

A análise a seguir considera N como o número total de ações, u o número de clientes, c o tamanho da carteira do cliente consultado, w a janela de cotações, e m o número de métricas utilizadas na consulta.

Operação M (Configuração): Leitura e armazenamento das métricas globais. Custo $O(k)$, onde $k \leq 4$ é o número de métricas definidas. Na prática, $O(1)$.

Operação A (Cadastro de Ação): Verificação de capacidade e eventual redimensionamento do vetor de ações. O acesso por índice é $O(1)$. O redimensionamento, quando ocorre, tem custo $O(N)$, mas o custo **amortizado** por inserção é $O(1)$, pois a capacidade dobra a cada redimensionamento — o custo total de n inserções é $O(n)$.

Operação U (Cadastro de Cliente): Igual à operação A. Custo amortizado $O(1)$.

Operação P (Cotação): Adiciona um preço ao vetor dinâmico interno da ação identificada. Custo amortizado $O(1)$.

Operação B (Compra): Inserção do ID da ação no vetor de carteira do cliente. Custo amortizado $O(1)$.

Operação V (Venda): Busca linear pelo ID da ação na carteira: $O(c)$. Remoção por substituição com o último elemento: $O(1)$. Custo total: $O(c)$.

Operação Q (Consulta): Para cada uma das m métricas da consulta realiza-se:

1. Cálculo da métrica para todas as N ações: $O(N \cdot w)$;
2. Ordenação via QuickSort: $O(N \log N)$ no caso médio;
3. Distribuição dos pontos ponderados: $O(N)$.

Ao final, monta-se e ordena-se o ranking local da carteira: $O(c \log c)$. O custo total por consulta é:

$$O(m \cdot N(w + \log N) + c \log c)$$

Como $c \leq N$ e $\log c \leq \log N$, o termo dominante é $O(m \cdot N(w + \log N))$.

A Tabela 1 resume os custos de cada operação nas duas estratégias de ordenação (Sob Demanda e Imediata), a escolhida para o código, considerada mais vantajosa em um cenário em que existem mais inserções e atualizações de cotação do que consultas foi a Sob demanda:

Operação	Sob Demanda	Imediata
A, U (cadastro)	$O(1)$ amortizado	$O(1)$ amortizado
P (cotação)	$O(1)$ amortizado	$O(k \cdot N(w + \log N))$
B (compra)	$O(1)$ amortizado	$O(1)$ amortizado
V (venda)	$O(c)$	$O(c)$
Q (consulta)	$O(m \cdot N(w + \log N))$	$O(c \log c)$

Tabela 1: Complexidade de tempo das operações nas duas estratégias. N : número de ações; m : métricas na consulta; w : janela; c : tamanho da carteira; k : métricas globais ativas.

3.2 Complexidade de Espaço

O programa mantém em memória:

- Um vetor contíguo de N objetos `Acao`, cada um com um vetor interno de até H cotações históricas: $O(N \cdot H)$ no total.
- Um vetor contíguo de u objetos `Cliente`, cada um com um vetor de IDs de ações: $O(\sum_{i=1}^u c_i)$, onde c_i é o tamanho da carteira do cliente i .
- Arrays temporários de `ParOrdenacao` durante as consultas: $O(N)$ para o ranking global e $O(c)$ para o ranking local.

O espaço total é dominado pelo histórico de cotações: $O(N \cdot H + \sum c_i)$.

4 Estratégias de Robustez

Para garantir o funcionamento correto mesmo em entradas extremas, foram adotadas as seguintes estratégias de programação defensiva:

1. **Redimensionamento seguro dos vetores principais:** Ao redimensionar os vetores de ações e clientes no `main`, a nova capacidade é calculada como $2 \cdot \text{cap_atual} + \text{id}$, e não simplesmente $2 \cdot \text{cap_atual}$. Isso garante que IDs arbitrariamente grandes sejam acomodados em um único redimensionamento, evitando alocações sucessivas. Para os vetores internos das classes (`cotacoes` em `Acao` e `carteira` em `Cliente`), o redimensionamento por fator de dois é suficiente, pois esses vetores crescem elemento a elemento e não estão sujeitos a saltos de índice.
2. **Semântica de cópia explícita:** As classes `Acao` e `Cliente` implementam **construtor de cópia** e **operador de atribuição** (`operator=`) explícitos. Sem esses métodos, o redimensionamento dos vetores principais — que copia objetos com `novo_vetor[i] = vetor[i]` — produziria uma *cópia rasa* dos ponteiros internos (`cotacoes*` e `carteira*`). Ao liberar o vetor antigo com `delete[]`, esses ponteiros se tornariam inválidos, apontando pro nada e causando comportamento indefinido silencioso. A implementação explícita garante que cada objeto redimensionado possua sua própria cópia independente dos dados.
3. **Proteção contra carteira vazia:** O bloco de impressão da consulta `Q` só é executado se `num_na_carteira > 0`, evitando alocação e acesso a arrays de tamanho zero.
4. **Limitação da quantidade impressa:** A variável `qtd` garante que nunca se tente imprimir mais ações do que existem na carteira, cobrindo o caso em que n excede o tamanho atual da carteira do cliente — conforme exigido pelo enunciado.
5. **Zeragem explícita do vetor de pontuações:** O array `pontuacoes_globais` é alocado com `new double[N]()`, cuja sintaxe com parênteses vazio zero-inicializa todos os elementos em C++11. Isso elimina a possibilidade de somar lixo de memória às pontuações calculadas.

6. **Liberação garantida de memória:** Todos os arrays temporários alocados dentro do bloco da consulta Q (`pontuacoes_globais`, `ranking_temp`, `metricas_consulta`, `pesos_consulta` e `carteira_rankeada`) são liberados com `delete[]` ao final do bloco, inclusive no caminho de carteira vazia. Os vetores principais são liberados ao final do `main`.

5 Análise Experimental

O objetivo central da análise experimental é avaliar quantitativamente a diferença de desempenho entre as estratégias **Sob Demanda** e **Imediata**, observando na prática o impacto de diferentes perfis de carga e validando os conceitos teóricos de crescimento assintótico estudados em aula.

5.1 Ambiente e Metodologia de Testes

Os testes foram realizados em um ambiente Linux (Ubuntu 22.04), utilizando o compilador `g++`. Para as medições temporais, utilizou-se a biblioteca `<chrono>`, permitindo aferições com precisão de milissegundos.

Os conjuntos de dados foram gerados de forma sintética por um gerador em Python, com uma seed fixa para garantir a reprodutibilidade. Nos experimentos realizados, considerou-se um único cliente ($u = 1$), com consultas utilizando $m = 2$ métricas ponderadas simultaneamente. Nos cenários em que o tamanho da janela histórica não era a variável independente, o valor de w foi fixado em 20. O número de clientes (u) não foi variado experimentalmente, pois o custo das operações de consulta depende predominantemente do número de ações (N) e do tamanho da carteira consultada (c), e não do número total de clientes no sistema. A análise foi dividida em três cenários exploratórios:

- **Cenário 1 (Escalabilidade Global):** Variação do número de ações (N) de 500 a 5.000, com o tamanho da carteira fixado em 50% de N .
- **Cenário 2 (Proporção de Operações):** Fixando o volume total de operações de cotação em 2.000, variou-se a frequência relativa das consultas (Q). Este cenário simula a transição de um ambiente de escrita intensiva para um ambiente de leitura intensiva.
- **Cenário 3 (Variáveis Secundárias):** Neste cenário, analisou-se separadamente o impacto de dois fatores: o tamanho da janela de cálculo (w), que determina quantas cotações passadas são usadas nas métricas, e o tamanho da carteira do cliente, medido como a porcentagem de ações do mercado que ele possui.

5.2 Resultados e Discussão

A Figura 2 consolida os resultados obtidos nos três cenários descritos, permitindo uma análise holística do comportamento das estratégias.

Análise Experimental: Estratégias de Ordenação de Carteiras

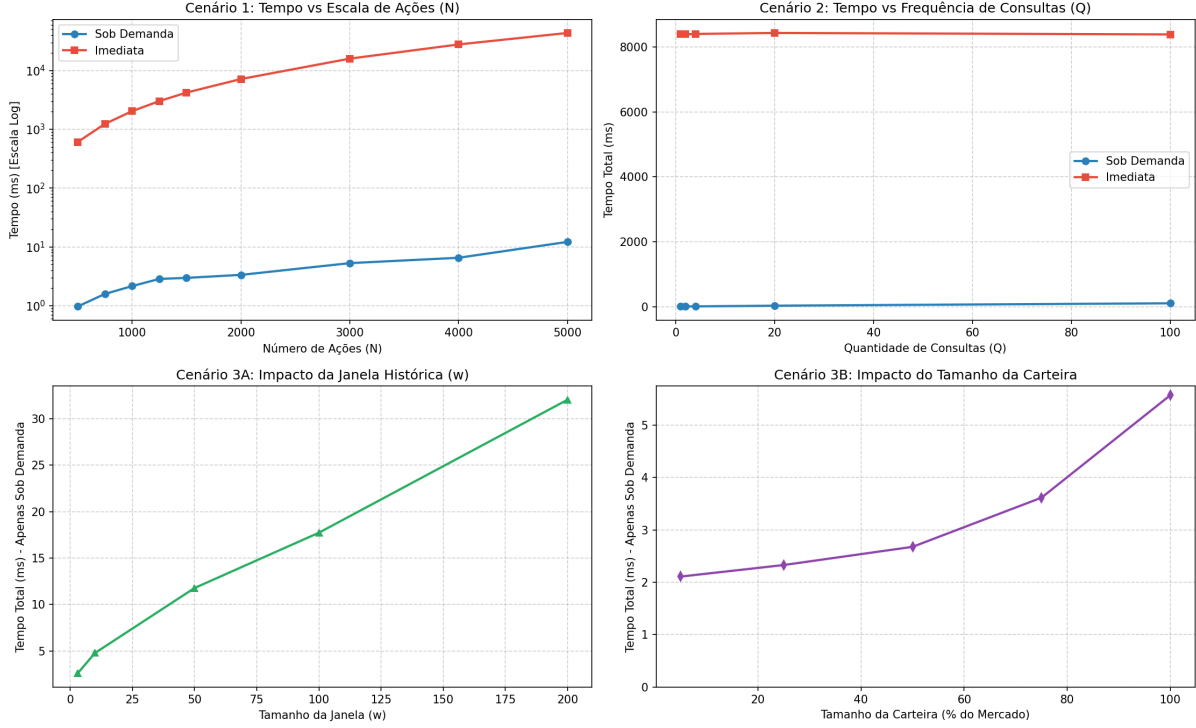


Figura 2: Pannel da Análise Experimental. O quadrante superior esquerdo evidencia a escalabilidade em função de N (escala logarítmica). O quadrante superior direito demonstra a sensibilidade à frequência relativa de consultas. Os quadrantes inferiores isolam a Estratégia Sob Demanda para comprovar o impacto da janela w e do tamanho da carteira.

Cenário 1: Impacto da Escalabilidade Global (N)

Conforme demonstrado no gráfico superior esquerdo da Figura 2, a Estratégia Imediata apresenta um tempo de execução significativamente maior, atingindo ordens de magnitude superiores à Estratégia Sob Demanda conforme N cresce. Esse comportamento está alinhado com o fato de que, na Estratégia Imediata, cada atualização de preço acarreta reordenações globais de custo $O(k \cdot N \log N)$, enquanto na Estratégia Sob Demanda esse custo é concentrado apenas nas consultas. O uso da escala logarítmica no eixo Y é essencial para visualizar a curva de eficiência da Estratégia Sob Demanda frente ao custo elevado da Estratégia Imediata.

Cenário 2: Frequência Relativa de Consultas (Q)

O gráfico superior direito ilustra o impacto da mudança no mix de operações. Na Estratégia Sob Demanda, o tempo de processamento cresce aproximadamente de forma linear em função do número de consultas, dado que o total de operações é mantido constante e cada consulta exige uma ordenação completa. Na Estratégia Imediata, a carga computacional é dominada pelo volume fixo de cotações (P); contudo, percebe-se que ela torna-se relativamente mais competitiva à medida que a densidade de consultas aumenta, embora ainda apresente um tempo de execução absoluto significativamente superior no cenário de alta frequência de dados testado.

Cenário 3: Janela Histórica (w) e Tamanho da Carteira

Nos gráficos inferiores, foi isolada a Estratégia Sob Demanda para análise de sensibilidade.

Observou-se que o aumento da janela (Cenário 3A) impõe um crescimento linear em relação a w no tempo de execução, refletindo o custo de percorrer as w cotações utilizadas no cálculo das métricas. Já a variação do tamanho da carteira (Cenário 3B) demonstrou um impacto de baixa magnitude na carga computacional global. Isso evidencia que a complexidade de ordenação global das N ações domina o tempo de resposta, tornando a ordenação local da carteira pouco relevante no tempo total.

Veredito:

Os experimentos validam que a **Estratégia Sob Demanda** evita reordenações desnecessárias e é categoricamente superior em ambientes voláteis (alta frequência de cotações). A Estratégia Imediata mostra-se ineficiente para este volume de dados, sendo indicada apenas para sistemas onde as cotações são estáticas e o volume de consultas sequenciais justifica o alto custo de atualização prévia.

6 Conclusão

Este trabalho consistiu no desenvolvimento de um sistema para análise dinâmica de carteiras de ações, implementado do zero em C++11, sem o uso de estruturas da biblioteca padrão. Construir vetores dinâmicos, gerenciar memória manualmente e implementar o próprio algoritmo de ordenação foram etapas que, no início, pareceram trabalhosas, mas que ao longo do desenvolvimento revelaram o quanto a STL abstrai decisões que têm impacto direto na performance.

A parte mais interessante do trabalho foi perceber, na prática, que não existe uma estratégia algorítmica universalmente melhor. A Estratégia Sob Demanda foi claramente superior no cenário testado — com muitas cotações e poucas consultas — porque concentra o custo computacional exatamente onde ele é necessário. A Estratégia Imediata, por sua vez, cobrou caro por cada operação, mesmo nas execuções em que nenhuma consulta foi feita em seguida, e isso justificou a necessidade de um desenvolvedor se atentar muito detalhadamente sobre a complexidade de cada etapa do seu código e o uso final de cada função criada, pra tomar as melhores decisões de implementação.

Outro aprendizado que ficou marcado foi o da importância de um gerenciamento robusto de memória, pois algo que não falha em pequenas entradas pode se tornar desastroso em grandes entradas, então cuidar de ponteiros apontando pro vazio e redimensionamento de arrays é importantíssimo.

Por fim, foi essencial ver que as curvas experimentais confirmaram o que a análise assintótica previa: o crescimento $O(m \cdot N(w + \log N))$ da Estratégia Sob Demanda e o crescimento proporcional ao número de operações P da Estratégia Imediata apareceram exatamente nos gráficos. Isso tornou a notação assintótica algo concreto e que causa impacto real no programa, e não apenas uma fórmula do slide.

7 Bibliografia

1. BHARGAVA, Aditya. **Entendendo Algoritmos**: Um Guia Ilustrado para Programadores e Outros Curiosos. São Paulo: Novatec, 2017.
2. CORMEN, Thomas H. et al. **Algoritmos**: Teoria e Prática. 3. ed. Rio de Janeiro: Elsevier, 2012. (Tradução de *Introduction to Algorithms*, 3^a ed., MIT Press, 2009).
3. CPPREFERENCE. **cppreference.com**. Documentação de referência técnica para as linguagens C e C++. Repositório utilizado para consulta de definições da biblioteca padrão e suporte à implementação conforme o padrão C++11. Disponível em: <https://cppreference.com/>.
4. ZIVIANI, Nivio. **Projeto de Algoritmos**: com implementações em Pascal e C. 3. ed. São Paulo: Cengage Learning, 2011. (Fonte de consulta para a implementação clássica do QuickSort.)
5. GOOGLE. **Gemini** (Versão 3.1 Pro). Assistente virtual de inteligência artificial utilizado para suporte na depuração de lógica em C++ e configuração de ambiente de testes. Mountain View, 2024. Disponível em: <https://gemini.google.com>.
6. ANTHROPIC. **Claude** (Versão Sonnet 4). Assistente virtual de inteligência artificial utilizado para consulta de conceitos teóricos, depuração, configuração de ambiente de testes e revisão da documentação. San Francisco, 2025. Disponível em: <https://claude.ai>.